INFORME TÉCNICO

Entrenamiento, Optimización y Regularización Aplicada en un Perceptrón Multicapa para Clasificación Multiclase sobre MINST

Santiago Lozano

FUNDAMENTOS DE REDES NEURONALES

20 de septiembre, 2026

1. Modelo base y problema identificado

En el desarrollo anterior del proyecto se trabajó con un conjunto de datos artificial compuesto por puntos distribuidos en forma de anillos, pertenecientes a dos clases distintas en un plano cartesiano. Ese problema permitió justificar el uso de un perceptrón multicapa (MLP) frente a un modelo lineal, dado que las clases no eran separables linealmente. Para esta nueva etapa se buscó acercar el ejercicio a un escenario más representativo del aprendizaje profundo, por lo que se decidió aumentar simultáneamente la complejidad del modelo y la del conjunto de datos utilizado.

Para ello se adoptó el conjunto de datos MNIST, compuesto por 70.000 imágenes de dígitos escritos a mano (0 a 9), cada una de 28 × 28 píxeles. Tras normalizar los valores de los píxeles entre 0 y 1 y aplanar cada imagen en un vector de 784 características, el conjunto completo se dividió en un 80% de entrenamiento y un 20% de prueba de forma estratificada, obteniendo 56.000 imágenes para entrenamiento y 14.000 para prueba.

Este cambio de conjunto de datos introdujo un nuevo tipo de problema: mientras que el ejercicio anterior era de clasificación binaria, MNIST exige distinguir entre diez categorías simultáneamente. Esto obligó a modificar tres elementos del modelo utilizado previamente. Primero, la capa de salida dejó de usar la función sigmoide, adecuada para dos clases, y pasó a utilizar softmax, que convierte las salidas de la red en una distribución de probabilidades sobre las diez categorías, asignando la predicción final a la de mayor probabilidad. Segundo, la función de pérdida dejó de ser una entropía cruzada binaria y pasó a ser una entropía cruzada categórica (multiclase), definida como:

$$L = -(1/N) \cdot \Sigma_n \Sigma_k \; y_\{n,k\} \cdot \log(\hat{y}_\{n,k\})$$

donde N es el número de muestras del lote evaluado, K es el número de clases (diez, en este caso), $y_\{n,k\}$ es la etiqueta real codificada en formato one-hot y $\hat{y}_\{n,k\}$ es la probabilidad predicha por la capa softmax para la clase k. Tercero, y como consecuencia directa del cambio de función de pérdida y de activación de salida, fue necesario ajustar la derivada utilizada en la retropropagación del error, así como codificar las etiquetas originales (valores de 0 a 9) mediante one-hot encoding, de forma que cada una de las diez salidas de la red pudiera compararse directamente con la clase correspondiente.

Con estos ajustes se estableció un modelo baseline compuesto por una capa de entrada de 784 neuronas, dos capas ocultas de 32 neuronas cada una con activación ReLU, y una capa de salida de 10 neuronas con activación softmax. Esta arquitectura se planteó deliberadamente pequeña, con el fin de mantener costos de procesamiento moderados y contar con una referencia clara sobre la cual medir el efecto de las modificaciones posteriores, más que con el objetivo de maximizar el desempeño desde el inicio.

Entrenado durante 30 épocas con descenso de gradiente estocástico puro (tamaño de lote de una muestra) y una tasa de aprendizaje de 0.01, el modelo baseline alcanzó una accuracy de 96.50% y un F1-score ponderado de 0.9649 sobre el conjunto de prueba. Sin embargo, al observar la evolución de la pérdida por época (véase Anexo A), se identificó el problema central que motiva el resto del informe: mientras que la pérdida de entrenamiento descendió de

forma consistente hasta 0.0563 en la época 30, la pérdida de prueba dejó de mejorar tempranamente y terminó en 0.1894, es decir, en un orden de magnitud distinto al de entrenamiento. Esta brecha creciente entre ambas curvas es el síntoma característico de overfitting: el modelo continúa ajustándose a los datos de entrenamiento, pero pierde capacidad de generalizar sobre datos que no ha visto. Identificado este problema, el diseño experimental que se describe a continuación busca intervenir de manera controlada sobre la arquitectura, la regularización y el algoritmo de optimización para mejorar tanto el desempeño como la estabilidad del entrenamiento.

2. Diseño experimental: decisiones técnicas implementadas y justificación

A partir del modelo base y del problema de overfitting identificado, se diseñaron cuatro experimentos controlados, cada uno orientado a evaluar el efecto de una decisión técnica específica frente a la misma referencia. En los tres primeros experimentos se modifica una única dimensión del problema (arquitectura, regularización u optimizador, respectivamente), manteniendo las demás condiciones lo más constantes posible frente al baseline, con el fin de aislar el efecto de cada intervención. El cuarto experimento combina las decisiones que mostraron mejores resultados en los experimentos individuales, para evaluar su efecto conjunto.

Experimento 1: Arquitecturas de red

El primer experimento evalúa cómo afectan al desempeño dos formas distintas de aumentar la capacidad del MLP: aumentar el ancho de las capas o aumentar su profundidad. La primera variante mantiene las dos capas ocultas del baseline, pero incrementa su tamaño a 64 neuronas cada una. La segunda variante conserva capas de 32 neuronas, pero agrega una tercera capa oculta, resultando en una arquitectura de 32-32-32. Adicionalmente, se evaluó una arquitectura aún más ancha, de 128 y 64 neuronas en las dos capas ocultas, para explorar el límite superior de esta estrategia.

Los resultados muestran que ampliar el ancho de las capas es más efectivo que aumentar la profundidad: la arquitectura de 64x64 alcanzó una accuracy de 97.33% (F1 ponderado de 0.9733), superando a la arquitectura de tres capas de 32 neuronas, que obtuvo 96.54% de accuracy (F1 de 0.9654). La arquitectura más ancha (128 y 64 neuronas) fue la que mejor desempeño individual alcanzó, con una accuracy de 97.8% y un F1 ponderado de 0.9785. No obstante, este beneficio tiene un costo computacional considerable: capas más anchas implican matrices de pesos más grandes y, por lo tanto, gradientes más costosos de calcular en cada actualización. Por esta razón, y para mantener tiempos de entrenamiento razonables en los experimentos siguientes, se adoptó la arquitectura de 64x64 neuronas como punto de partida para las pruebas de regularización y optimización, en lugar de la variante más ancha de 128x64 (véase Anexo B, curva de pérdida de la arquitectura 64x64).

Experimento 2: Regularización de parámetros

El segundo experimento aborda directamente el overfitting identificado en la sección anterior, mediante regularización de los pesos de la red. Esta técnica añade a la función de pérdida original un término de penalización relacionado con la magnitud de los pesos, de forma que el

modelo no solo busca minimizar el error de clasificación, sino también evitar que los parámetros crezcan de forma excesiva. En este experimento se implementó regularización L1, que penaliza la suma de los valores absolutos de los pesos de cada capa. La función de pérdida total queda entonces expresada como:

$$L_total = L + (\lambda / (2N)) \cdot \Sigma_l \Sigma_{ij} |(W_l)_{ij}|$$

donde λ es un nuevo hiperparámetro, generalmente entre 0 y 1, que determina la importancia relativa de la penalización: valores cercanos a 1 dan mucho peso al tamaño de los pesos, mientras que $\lambda = 0$ elimina por completo el término de regularización y devuelve la función de pérdida original. A diferencia de una penalización cuadrática, el gradiente de la penalización L1 depende únicamente del signo de cada peso y no de su magnitud, por lo que su efecto sobre la actualización de parámetros es constante independientemente de qué tan grande o pequeño sea el peso.

Sobre la arquitectura baseline de 32x32 neuronas se probaron inicialmente valores de λ del orden de 0.1 y 0.2, obteniendo resultados muy deficientes (accuracy cercana al 9.8%, prácticamente equivalente a una predicción aleatoria entre diez clases), ya que, al ser constante independientemente de la magnitud de los pesos, la penalización L1 dominaba por completo el cálculo del gradiente e impedía que el modelo aprendiera. Al reducir el orden de magnitud de λ a valores de 0.001, 0.01 y 0.05, el comportamiento mejoró sustancialmente: con $\lambda = 0.001$ se alcanzó una accuracy de 89.95% (F1 de 0.8993), con $\lambda = 0.01$ la accuracy bajó a 75.28% (F1 de 0.7468), y con $\lambda = 0.05$ el modelo volvió a colapsar a un desempeño cercano al azar (9.82% de accuracy). El mejor resultado, $\lambda = 0.001$, se mantuvo por debajo del 96.50% del modelo baseline sin regularizar; sin embargo, sus curvas de pérdida (véase Anexo C) muestran un comportamiento cualitativamente distinto: la pérdida de entrenamiento se mantiene más estable en lugar de descender agresivamente, y la pérdida de prueba, aunque más irregular, se ubica por debajo de la de entrenamiento, evidenciando el efecto buscado de la regularización sobre la generalización, incluso cuando la accuracy bruta no mejora frente al baseline en esta arquitectura particular.

Experimento 3: Algoritmos de optimización

El tercer experimento se enfoca en el algoritmo de optimización utilizado para actualizar los pesos durante el entrenamiento. Esta decisión se motivó por la variabilidad observada en la pérdida de entrenamiento de los experimentos anteriores, asociada a la magnitud igualmente variable de los gradientes calculados en cada paso. Los algoritmos de optimización buscan estabilizar este proceso, evitando tanto gradientes que exploten como gradientes que se desvanezcan.

Manteniendo la arquitectura baseline de 32x32 neuronas y entrenando ahora con mini-lotes de 32 muestras (en lugar de una sola muestra por actualización, como en los experimentos anteriores), se compararon tres estrategias: descenso de gradiente estocástico estándar (SGD), SGD con momentum, y Adam. La actualización general de parámetros en el descenso de gradiente puede expresarse como:

$$\theta_\{t+1\} = \theta_t - \eta \cdot \nabla\theta\, J(\theta_t)$$

donde θ representa los parámetros del modelo (pesos y sesgos), η es la tasa de aprendizaje y ∇θ J(θ_t) es el gradiente de la función de pérdida respecto a esos parámetros. La variante con momentum incorpora una memoria de las actualizaciones anteriores mediante un término de velocidad v, ponderado por un coeficiente β (en este caso, 0.9):

$$v_t = \beta \cdot v_\{t-1\} - \eta \cdot \nabla\theta\, J(\theta_t) \qquad \theta_\{t+1\} = \theta_t + v_t$$

Adam, por su parte, ajusta de forma adaptativa el tamaño del paso para cada parámetro individual a partir de estimaciones de primer y segundo momento del gradiente, con corrección de sesgo; dado que esta formulación involucra varios términos adicionales no cubiertos en detalle en el taller, no se incluye su expresión completa, y su implementación se apoyó en los hiperparámetros estándar ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1\times10^{-8}$).

Los resultados muestran que, sobre esta arquitectura pequeña, el SGD por lotes fue el que mejor desempeño obtuvo, con una accuracy de 96.64% (F1 de 0.9663), seguido muy de cerca por momentum, con 96.59% (F1 de 0.9658), y por Adam, con 95.68% (F1 de 0.9568). Al observar las curvas de pérdida (véase Anexo D), se aprecia que con SGD el entrenamiento y la prueba se mantienen relativamente cercanos, mientras que con momentum y, en mayor medida, con Adam, la pérdida de prueba se separa más de la de entrenamiento. Esto sugiere que, para un problema de clasificación relativamente sencillo como el aquí planteado y con una arquitectura pequeña, un gradiente sin ajustes adicionales resulta suficiente, mientras que los mecanismos adaptativos de momentum y Adam no logran aportar una ventaja clara y, en cambio, introducen mayor variabilidad entre entrenamiento y prueba.

Experimento 4: Técnicas combinadas

El último experimento combina las decisiones evaluadas anteriormente sobre una arquitectura de mayor capacidad. Se utilizó la arquitectura de 64x64 neuronas (la de mejor relación desempeño-costo del Experimento 1) junto con cada uno de los tres optimizadores (SGD, momentum y Adam). A diferencia del Experimento 2, en este caso se utilizó regularización L2 (Ridge Regression) en lugar de L1, ya que L2 penaliza el cuadrado de la magnitud de los pesos, reduciéndolos de forma proporcional a su tamaño sin forzarlos a cero. Esta alternativa resulta más adecuada sobre una arquitectura más ancha, considerando que el Experimento 1 mostró que mantener una mayor cantidad de neuronas activas por capa favorece el desempeño del modelo. La función de pérdida total en este caso corresponde a:

$$L_total = L + (\lambda / (2N)) \cdot \Sigma_l \Sigma_{ij} (W_l)_{ij}^2$$

A diferencia de la penalización L1 del Experimento 2, aquí cada peso se penaliza de forma proporcional a su valor al cuadrado, por lo que pesos pequeños reciben una penalización aún menor y no se ven forzados hasta cero. Se evaluó cada optimizador tanto sin regularización como con esta penalización L2, usando $\lambda = 0.01$, uno de los órdenes de magnitud que en el Experimento 2 evitó el colapso del modelo. En total se entrenaron y evaluaron seis configuraciones combinadas.

La mejor configuración combinada, y la de mejor desempeño de todo el informe, fue la arquitectura de 64x64 neuronas con optimizador momentum y sin regularización adicional, con una accuracy de 97.87% y un F1-score ponderado de 0.9787 (véase Anexo E). Le siguió la

misma arquitectura con momentum y regularización L2 (λ = 0.01), con 97.45% de accuracy (F1 de 0.9745). Estos resultados se presentan de forma comparativa en la sección siguiente.

3. Resultados comparativos: curvas, métricas y evidencia cuantitativa

La Tabla 1 resume el mejor resultado obtenido en cada uno de los cuatro experimentos, permitiendo comparar directamente el efecto de cada tipo de intervención frente al modelo baseline.

Configuración	Accuracy	F1 (weighted)
Modelo baseline (32x32, SGD puro)	96.50%	0.9649
Exp. 1 – Mejor arquitectura (128x64)	97.80%	0.9785
Exp. 1 – Mejor arquitectura graficada (64x64)	97.33%	0.9733
Exp. 2 – Mejor regularización (32x32, L1 λ=0.001)	89.95%	0.8993
Exp. 3 – Mejor optimizador (32x32, SGD por lotes)	96.64%	0.9663
Exp. 4 – Mejor combinación (64x64, Momentum, sin reg.)	97.87%	0.9787

Sobre las seis configuraciones combinadas del Experimento 4, la Figura 1 presenta, con el eje vertical truncado entre 0.960 y 1.000 para facilitar la comparación visual entre resultados numéricamente cercanos, tanto la accuracy como el F1-score ponderado de cada configuración.

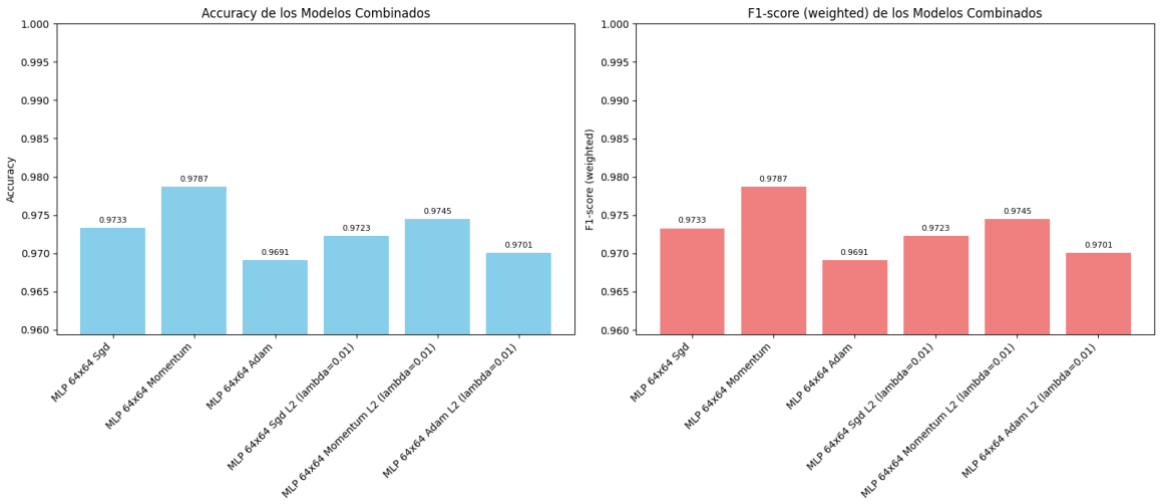


Figura 1. Accuracy y F1-score (weighted) de las seis configuraciones combinadas del Experimento 4, sobre eje truncado (0.960–1.000).

La figura permite observar tres patrones. Primero, momentum es consistentemente el optimizador de mejor desempeño en esta arquitectura, tanto con regularización como sin ella, y por un margen claro frente a Adam. Segundo, la regularización L2 con λ = 0.01 no mejora el

resultado de la configuración sin regularizar en el caso de momentum ni en el de SGD; en ambos casos la accuracy y el F1 disminuyen levemente al añadir el término de penalización, mientras que en Adam el efecto es marginal y ligeramente positivo (de 96.91% a 97.01% de accuracy). Tercero, incluso la peor de las seis configuraciones combinadas (Adam sin regularización, 96.91%) supera el resultado del modelo baseline (96.50%), lo que confirma que el cambio de arquitectura de 32x32 a 64x64 aporta una mejora de base independiente del optimizador o la regularización utilizados.

4. Análisis crítico de las mejoras obtenidas y conclusiones técnicas

De los tres tipos de intervención evaluados, la arquitectura resultó ser la variable con mayor impacto sobre el desempeño del modelo. Ampliar el ancho de las capas ocultas (de 32 a 64 e incluso a 128 neuronas) generó mejoras consistentes de accuracy tanto de forma aislada (Experimento 1) como en combinación con distintos optimizadores (Experimento 4), donde incluso la peor combinación sobre la arquitectura 64x64 superó al mejor resultado obtenido nunca sobre la arquitectura 32x32 sin regularizar. Aumentar la profundidad en lugar del ancho (arquitectura 32-32-32) resultó menos efectivo que ensanchar las capas existentes. Esta ganancia de desempeño, sin embargo, no es gratuita: capas más anchas incrementan directamente el tamaño de las matrices de pesos y, con ello, el costo de calcular y actualizar los gradientes en cada paso de entrenamiento, lo que explica por qué no se llevó la arquitectura más ancha (128x64) a los experimentos de regularización y optimización, priorizando tiempos de entrenamiento manejables.

El efecto de la regularización resultó ser el más matizado de los tres tipos de intervención. En la arquitectura pequeña (32x32), con penalización L1, cualquier valor de λ probado redujo la accuracy frente al modelo sin regularizar, y valores demasiado altos (0.1, 0.2, 0.05) llevaron al modelo a un colapso cercano al desempeño aleatorio, un efecto consistente con el hecho de que la penalización L1 empuja los pesos con una magnitud constante independiente de su tamaño, dominando el gradiente cuando λ es grande. En la arquitectura más grande (64x64), con penalización L2, el patrón general se repitió: la configuración sin regularizar superó a su contraparte regularizada tanto con SGD como con momentum, y solo mostró una mejora marginal con Adam. En ningún caso evaluado la regularización produjo la mejor accuracy absoluta de su grupo; su valor se observa, en cambio, en el comportamiento de las curvas de pérdida, donde el mejor λ del Experimento 2 mostró una relación más estable y controlada entre entrenamiento y prueba que el modelo sin regularizar, aun a costa de una accuracy menor. Esto confirma que la regularización, en este problema, actúa como un mecanismo de control de la generalización que debe calibrarse cuidadosamente en función de la capacidad del modelo y del tipo de penalización utilizada, y no como una fuente directa de mejores métricas de clasificación.

En cuanto a los algoritmos de optimización, el resultado más relevante es que el mejor optimizador dependió de la capacidad de la arquitectura sobre la que se aplicó: en la red pequeña (32x32) el SGD por lotes, sin ningún mecanismo adicional, fue ligeramente superior a momentum y claramente superior a Adam; en la red más grande (64x64), en cambio, momentum fue el optimizador con mejor desempeño de todo el informe. Esto es consistente con la función que cumple momentum, al acumular información de gradientes pasados para

suavizar y acelerar la convergencia: su beneficio se vuelve más relevante cuanto más compleja es la superficie de optimización, como ocurre al aumentar el número de parámetros del modelo. Adam, por su parte, obtuvo el peor resultado en ambas arquitecturas, lo que indica que sus ajustes adaptativos por parámetro no son necesarios para un problema de clasificación de esta naturaleza y, dentro del número fijo de épocas utilizado, no logran converger a una solución mejor que la de los métodos más simples.

En conclusión, la mejor configuración global obtenida en este taller corresponde a un MLP con dos capas ocultas de 64 neuronas cada una, entrenado con el optimizador momentum ($\beta = 0.9$) y sin regularización adicional, alcanzando una accuracy de 97.87% y un F1-score ponderado de 0.9787 sobre el conjunto de prueba de MNIST. Esta configuración representa una mejora clara frente al modelo baseline (96.50% de accuracy), obtenida principalmente a través del ensanchamiento de las capas ocultas y, en segundo lugar, mediante la elección de un optimizador con memoria de gradientes anteriores. La regularización, si bien fundamentada teóricamente para controlar el overfitting identificado en la Sección 1, no se tradujo en este caso en una mejora de las métricas de clasificación, y su principal aporte fue evidenciar, a través de las curvas de pérdida, un comportamiento de entrenamiento más controlado cuando se calibra con un λ adecuado a la capacidad del modelo.

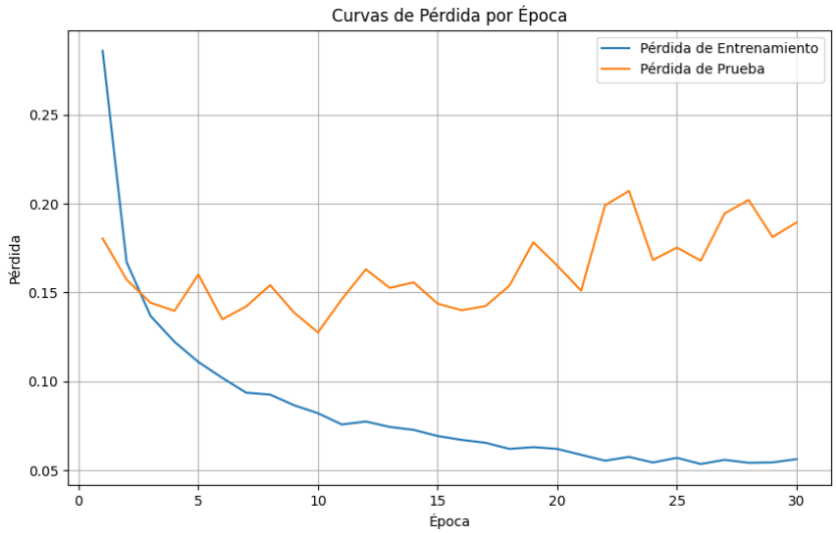
Nota IA

Este texto fue corregido con el apoyo de Claude (IA de Anthropic) como herramienta de asistencia en la redacción.

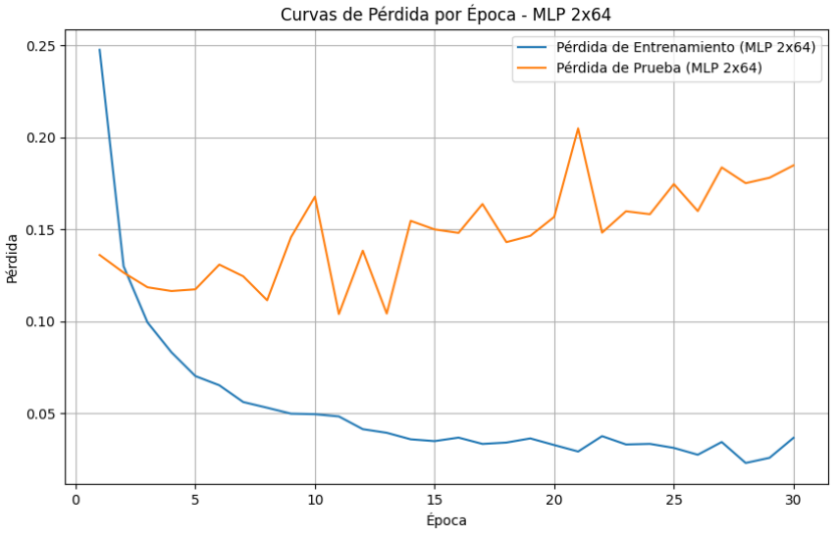
Anexos

A continuación se presentan las curvas de pérdida (entrenamiento y prueba) por época correspondientes al modelo baseline y a la configuración de mejor desempeño de cada uno de los cuatro experimentos descritos en este informe.

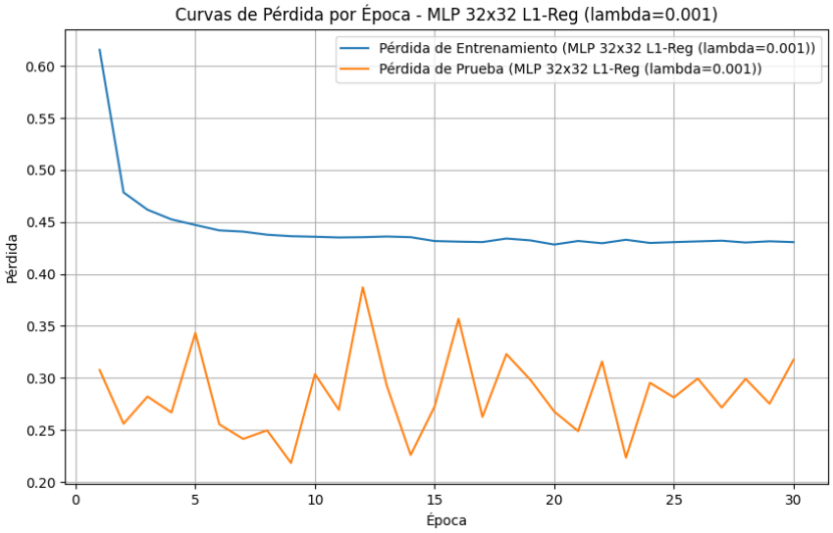
Anexo A. Curva de pérdida del modelo baseline (MLP 32x32, SGD puro)



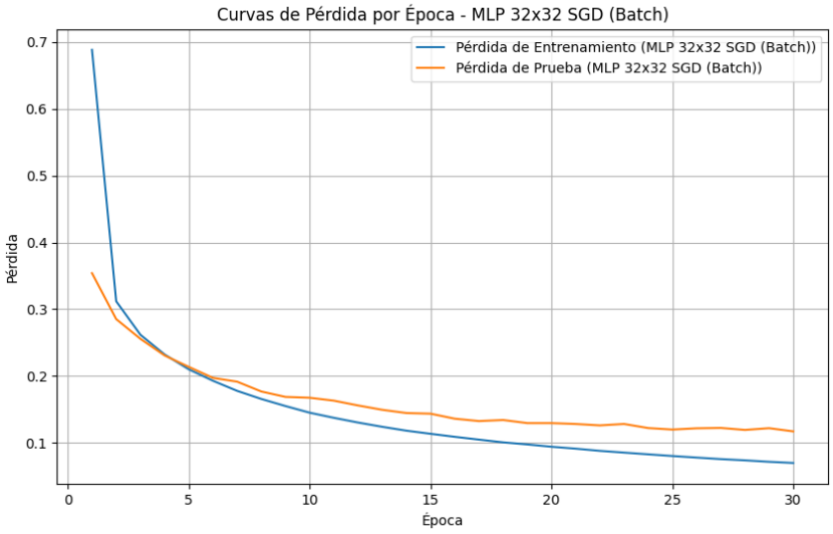
Anexo B. Curva de pérdida — Experimento 1, arquitectura 64x64



Anexo C. Curva de pérdida — Experimento 2, regularización L1 (32x32, λ = 0.001)



Anexo D. Curva de pérdida — Experimento 3, SGD por lotes (32x32)



Anexo E. Curva de pérdida — Experimento 4, mejor combinación (64x64, Momentum, sin regularización)

